StateShift

Move a running workload between machines at a step boundary — without restarting it, and without repeating committed work.

Internal brief for the deck. Everything in this document is either **built and measured** or explicitly labelled **planned**. Nothing here is aspirational padding — a judge can ask about any number on a slide and this file contains the answer.

CORE IN GO	COMMIT	BEST MEASURED PAUSE
3,254	6a16485	3 ms
lines · 19 files · 6 packages	github.com/guneettoppo/stateshift	loopback, 12 KiB state

DOCUMENT TYPE · technical brief + slide-by-slide deck blueprint
AUDIENCE · deck owner (slides, visuals, narration) and the build team
STATUS · core engine built and tested · cloud wiring not yet executed
RULE · if a claim is not in section 11, it does not go on a slide

How to use this before you open PowerPoint

You are building a **5-minute pitch deck** for a 24-hour hackathon with a technical judging panel and several sponsor prize categories. This file gives you the argument, the numbers and a slide-by-slide blueprint. Follow the blueprint, keep the numbers, cut nothing that is marked **must appear**.

The three jobs the deck has to do

1. **Land the problem in 15 seconds.** Not "distributed systems are hard" — a concrete, money-shaped failure: a long-running job dies and you start it from zero.
2. **Prove it is not a concept.** The project exists, has a measured pause time, a measured dedupe ratio and a test that proves no work was repeated. Most competing decks cannot put a number on their central claim.
3. **Look honest about what it is not.** One slide that says "we migrate application state, not kernel memory" buys more credibility than any feature list.

Reading order if you are short on time

SECTION	READ IT BECAUSE
2	The 60-second story. If you only memorise one page, this one.
12	The deck blueprint: 14 slides with copy, visuals and speaker lines.
13	Numbers cheat sheet — the figures you must say out loud without looking.
14	Judge Q&A bank. Rehearse the four bolded ones.
11	What is actually built vs planned. Never overclaim past this table.

> THE ONE RULE
>
> Every quantitative claim on a slide must trace to a row in section 11. If you want a number that is not there, ask the build lead — do not estimate it, do not "round up" a benchmark, and never present loopback numbers as wide-area numbers.

What this project is, in one paragraph (steal this verbatim)

StateShift is a control plane for moving a **live workload** from one machine to another. A job is expressed as a state machine; every step commits its effects to an exactly-once op log and its state to a content-addressed checkpoint. To migrate, the source freezes at a step boundary, the checkpoint is transferred as a delta of only the chunks that changed, and an **epoch fence** in the database hands authority to the target in one atomic statement. The source can no longer commit — provably, not by timeout — so the destination resumes without repeating a single committed effect. We measure the pause and the bytes moved, and we show both on screen while it happens.

The story the deck tells

Act 1 — the failure everyone has lived through (15 s)

You start a long job. An hour in, the machine reboots, the container is rescheduled, or the spot instance is reclaimed. Everything since the last save is gone. You run it again and pay twice.

Three concrete shapes of this, all true and all easy to say out loud:

- **An LLM agent run.** 40 tool calls, $6 of tokens, and the box dies at call 31. Every committed call is already paid for; the restart pays again.
- **A simulation / batch compute.** Six hours of progress held in memory. Restarting means six hours.
- **A stateful session.** Unpersisted state, open handles, work in flight. There is no "just redeploy it" that does not lose the session.

Act 2 — why the standard answers are not answers (20 s)

STANDARD ANSWER	WHY IT MISSES
Redeploy / restart on the new node	Works only for stateless services. Loses in-memory progress and duplicates committed side effects.
Persist everything after each step	That is durable execution, and it keeps the job pinned to one machine. It does not move it, and it does not survive the machine.
CRIU / transparent process migration	The real thing, and it needs root, matching kernels, matching environments. In a 24-hour window it either works completely or you demo nothing.
VM live migration	Solves it at the hypervisor layer, which means you own the hypervisor. Not available to an app team.

Act 3 — what we actually built (25 s)

A middle path that is **honest, buildable and measurable**: migrate the workload's **explicit state** at step boundaries, with a database-enforced ownership handover so two nodes can never both be authoritative.

MEASURED PAUSE
3–7 ms
loopback · sub-second expected on WAN

SECOND CHECKPOINT
1 / 7
chunks re-sent (≈10% of bytes)

REPEATED WORK
0
ops == steps after migration

> THE SENTENCE THAT WINS THE ROOM
>
> "Two nodes can both think they own the job — but only one of them can commit, because ownership is a single database row, not a heartbeat."

Problem statement, in the hackathon's own language

The challenge we chose asks for a platform that migrates running workloads between machines. It lists the hard parts explicitly. Here is how the deck should map them — one row per requirement, because judges often grade against the brief's own checklist:

REQUIREMENT IN THE BRIEF	WHAT STATESHIFT DOES	WHERE IT IS PROVEN
Checkpointing capture enough state to resume	Step-boundary snapshot of the workload's state struct, serialised, chunked, hashed	`chunk + e2e tests`
Transfer efficiency avoid retransmitting unchanged state	Content-defined chunking + existence check; only novel chunks are uploaded	`1 of 7 chunks on 2nd checkpoint`
Consistency prevent two authoritative instances	Epoch fence: one guarded `UPDATE … WHERE epoch = $current`	`fence-contention test`
Network identity endpoints that depend on the original machine	Jobs are addressed only by logical ID through the coordinator; clients never learn a node address	`API surface`
Recovery decide source vs destination after a mid-move failure	Abort before the fence → source keeps running. Fail after the fence → authority returns with a new epoch. Every attempt recorded, including failures	`migrations table`
Demo pause duration + bytes moved, computation finishes	Live: pause in ms, bytes transferred, chunks sent/skipped, epoch before/after, op count	`dashboard, on screen`

Scope decision — say this out loud, early

We migrate explicit application state, not kernel state. That means no sockets, no file descriptors, no thread stacks. In exchange: no root, no kernel patches, works on any OS, and the failure modes are ordinary software failures instead of "the kernel was compiled without CONFIG_CHECKPOINT_RESTORE".

This is the same trade-off real systems make: Temporal, Restate and Netflix's Conductor all persist *application* state at step boundaries. We add the part they do not have — the ability to **move** the running job and measure the move.

Why this problem is worth 24 hours

- It is hard for the **right** reasons: semantics, coordination, serialisation, failure handling — not because it needs a big model or a big dataset.
- It produces **numbers**, and numbers are what separate a demo from a prototype.
- It is **demoable in one screen**: a job counter moving, a button, and the counter continuing on another node.
- Failure cases are **showable**, and showing a failed migration adjudicated correctly is more impressive than showing a happy path.

System architecture



Three roles, one binary

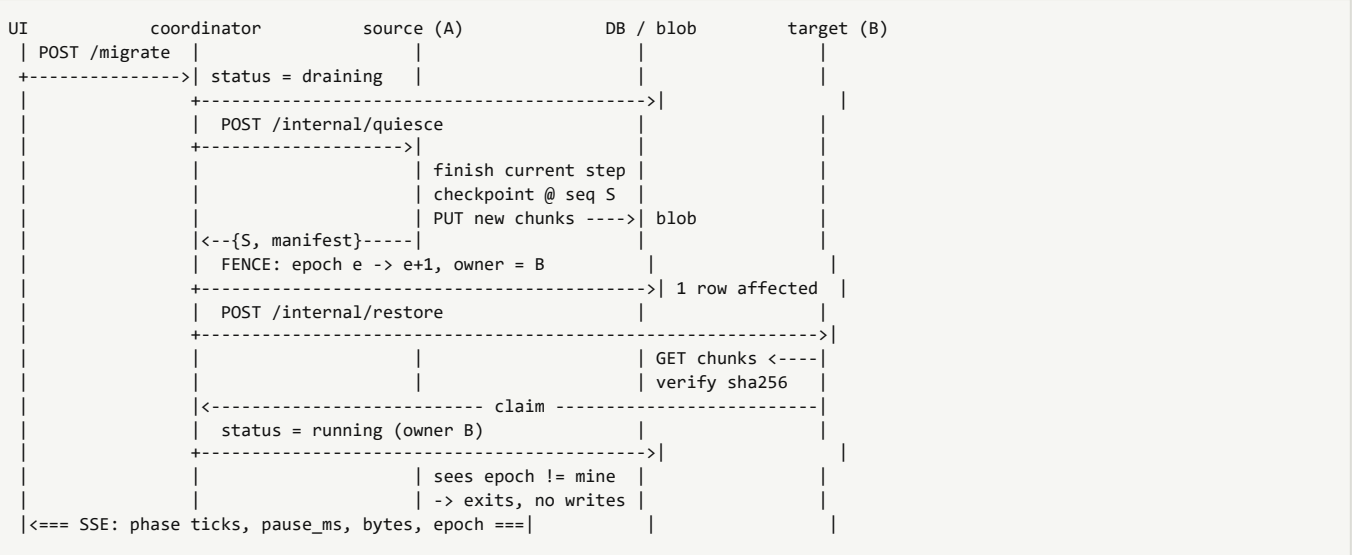
COMPONENT	RESPONSIBILITY	PACKAGE
Worker	Runs one job at a time per job ID. Commits a step, checkpoints on request, freezes, restores and resumes. Exposes `/internal/start`, `/quiesce`, `/restore`.	`internal/node`
Coordinator	Fleet registry, job creation, drives the handover, serves the dashboard. The only component the UI talks to.	`internal/coordinator`
Workload	The migratable program: `Init`, `Step`, `Checksum`. Ships `compute` and `agent`.	`internal/workload`
Store	Control-plane database. Two implementations with identical semantics: `mem` and `pg` (YugabyteDB YSQL / Postgres).	`internal/store`
Chunk store	Content-addressed blobs + manifests. Two implementations: filesystem and S3-compatible.	`internal/chunk, internal/blob`

Two design decisions worth a slide

1. **Jobs are never addressable on a node.** They exist at `/api/jobs/{id}` on the coordinator. Consumers hold a logical handle, and the coordinator resolves it to whichever node currently holds the epoch. That is how we answer "network identity" without pretending to move sockets: identities follow the job, not the machine.
2. **The database is the only authority.** Not the coordinator's memory, not a heartbeat, not a TTL. If the coordinator crashes mid-migration, the epoch row still names exactly one owner.

> DEPLOYMENT RULE (PUT THIS ON THE ARCHITECTURE SLIDE AS A FOOTNOTE)
>
> The control plane and the database must not live on either worker. If the database sits on the node you kill during the demo, the demo dies with it.

What actually happens when you press Migrate



The eleven steps, in words

#	STEP	WHY IT IS THERE
1	Coordinator marks the job `draining` (guarded by status)	Intent only. Authority is untouched, so a failure here is free.
2	Coordinator calls `/internal/quiesce` on the owner	The worker keeps its authority and finishes the step it is in.
3	Worker commits that final step, then stops at the boundary	A *draining* owner is still allowed to commit — otherwise every migration silently loses its last step.
4	Worker chunks the state, uploads only missing chunks, writes a manifest	This is the transfer-efficiency claim, and it is measurable.
5	Worker inserts a `checkpoints` row and replies with the sequence number	Now the state is durable somewhere other than the source's memory.
6	Coordinator performs the **fence**: epoch + 1, owner = B, status = restoring	The single point of truth. Must affect exactly one row or the migration aborts.
7	Coordinator calls `/internal/restore` on the target with the manifest	Target pulls only the chunks it needs and reassembles the state.
8	Target verifies the reassembled state against the hash the control plane committed	Blob and database must agree before anyone runs under a new epoch.
9	Target claims the job and flips status to `running`	Only now does the new owner begin executing.
10	Source observes the epoch changed and exits without side effects	Its next commit would have been rejected anyway; this is the polite version.
11	Coordinator computes pause = resumed − quiesced, records the migration row	The number on the slide is a measured interval, not an estimate.

Pause definition, so nobody argues on stage: from the moment the source freezes at a step boundary to the moment the target begins executing the next step. It includes the checkpoint upload, the fence, the download and the hash verification.

The epoch fence — the hardest claim in the project

Everything else is engineering. This is the part that is easy to get wrong and impossible to fake. Show the SQL. Judges who know distributed systems will recognise it, and judges who don't will understand "exactly one row can win".

```
-- ownership handover: exactly one caller can win this
UPDATE jobs
   SET epoch  = epoch + 1,
       owner  = $target,
       status = 'restoring'
 WHERE id    = $job
   AND epoch = $current;
-- rows affected must be 1, otherwise abort and leave the source running
```

Why a heartbeat is not enough

A heartbeat answers "is it alive?" with a guess, several hundred milliseconds late. During the gap both nodes believe they own the job and both write. The epoch answers "who may write?" with a database predicate that is evaluated at commit time, on the only copy of the truth.

Every write carries the epoch

```
-- a step commit is rejected the moment ownership moves
UPDATE jobs
   SET seq = $next, state = $state, state_hash = $hash, updated_at = now()
 WHERE id = $job AND epoch = $epoch AND seq = $from
   AND status IN ('running','draining');
-- 0 rows  =>  the caller is stale  =>  stop, do not retry, do not write
```

The rejection is the proof. A fenced-out node does not have to "notice" anything, and the target does not have to wait for a timeout to expire before it is safe.

> SAY THIS LINE ON STAGE
>
> "If both nodes could write, our demo would have shown double execution. It can't, because the second writer's `WHERE` clause no longer matches."

The subtle bug we found (and this is a good story to tell)

First version required `status = 'running'` in the commit predicate. That meant the moment the coordinator set `draining`, the source's in-flight step could no longer commit: the quiesce call hung and the migration timed out. Every migration would have silently lost one step — the exact failure the project exists to prevent.

Draining now means "start no new work", not "lose authority". There is a regression test specifically for it. On stage this costs 15 seconds and buys an enormous amount of credibility: it shows the team understands the difference between intent and authority.

Two mechanisms that make the numbers real

1 · Exactly-once effects, structurally

Each step produces zero or more effects with stable IDs. The effect log and the state advance are written in **one transaction**, so they can never disagree:

```
BEGIN;
  INSERT INTO oplog (job_id, seq, epoch, op_id, result)
       VALUES ($job, $seq, $epoch, $op_id, $result)
  ON CONFLICT (job_id, seq, op_id) DO NOTHING;

  UPDATE jobs SET seq = $next, state = $state, ...
   WHERE id = $job AND epoch = $epoch AND seq = $from;   -- 0 rows => stale, abort
COMMIT;
```

If a step is ever executed twice — a restart, a retry, a resurrected source — the second insert is a no-op. Duplication stops being a policy and becomes a schema constraint.

Proof on stage: after a mid-flight migration, the op log has exactly *N* rows for *N* steps. The test asserts it; the dashboard shows it.

2 · Delta transfer by content-defined chunking

PARAMETER	VALUE	MEANING
Chunk boundary	`gear hash, mask 0x3FF`	~1 KiB average, content-defined so boundaries survive edits
Min / max chunk	`256 B / 4 KiB`	Caps overhead on tiny states and pathological runs
Chunk identity	`sha256`	Content-addressed key; identical chunks are the same object
Dedupe test	`exists → skip`	Only chunks the store has never seen are uploaded
Manifest	`JSON`	Chunk list + total state hash, so a restore can verify itself

Measured (not modelled)

STATE SIZE

12.4 KiB

resident dataset + progress tail

CHUNKS PER CHECKPOINT

7

avg ≈ 1.7 KiB

2ND CHECKPOINT UPLOADS

1

6 skipped · 1,268 B of 12,377 B

≈ 10% of the bytes for the second snapshot. That is the "avoid retransmitting unchanged state" requirement, reduced to a single ratio you can put in a big font.

Why state shape matters (a good technical aside): the mutable counters must sit at the *end* of the serialised state and keep a fixed width. If a counter before a large blob changes length, every byte after it shifts, every chunk boundary moves, and dedupe collapses to zero. Our first version did exactly that and the metric was 0%. Immutable head, fixed-width tail.

What happens when the happy path breaks

This is the slide that separates a demo from a system. Every row below ends with exactly one authoritative owner.

FAILURE	WHAT THE SYSTEM DOES	WHAT THE AUDIENCE SEES
Source dies before the fence	Coordinator rolls the status back and restarts the source from its committed state	`job continues on A`
Source dies during checkpoint upload	No checkpoint row exists, so nothing is restorable — the retry re-uploads. No partial state is ever adopted	`migration aborts`
Target fails to restore (after the fence)	Authority returns to the source with another epoch bump; the source restarts from the last checkpoint	`outcome: failed-failover-back`
Both nodes alive, network partitioned	Only the epoch holder can commit; the other gets a stale-epoch rejection and exits	`no duplicate execution`
Restored state does not match the committed hash	Target refuses to resume under the new epoch — blob and database disagree, so nothing runs	`hard stop, not a guess`
Coordinator crashes mid-migration	The epoch row still names one owner; a restarted coordinator can read where it stopped	`no split brain`

Every attempt is recorded — including the failures

The `migrations` table stores from/to, epoch before and after, quiesce and restore timestamps, pause, bytes moved, chunks sent and skipped, and an outcome string: `ok`, `aborted-quiesce-failed`, `aborted-fence-lost`, `failed-failover-back`.

On stage, the valuable moment is showing a **failed** row and explaining what adjudicated it. Any team can show green rows; the red ones are the ones that prove the design.

> HONEST GAP — KNOW IT BEFORE A JUDGE FINDS IT
>
> Failure handling exists **inside a migration attempt**. A node that dies while it owns a job and nobody asked it to migrate is not yet auto-recovered by a watchdog; the job stalls at `running` with a dead owner. One reaper loop (owner heartbeat + epoch bump to a survivor) closes it. If asked, say exactly that — it is a known, scoped gap, not an oversight.

Technology choices and why they map to prizes

LAYER	CHOICE	WHY THIS ONE	FALLBACK IF IT FAILS
Language	**Go 1.24+**	No cgo, cross-compiles to Linux in one command, goroutines map cleanly onto a job loop	—
Transport	net/http + JSON, SSE for UI	No protobuf toolchain, readable on stage, nothing to debug at 3 AM	data plane alone can move to gRPC later
Control DB	**YugabyteDB Aeon (YSQL)**	Distributed SQL with a real fence predicate; also the title sponsor's track	`--db=pg` against Postgres
Blob store	**DO Spaces** (S3-compatible)	Content-addressed chunks, cheap, same region as the droplets	`--blob=fs` local dir
Model	**Gemini** (generateContent)	The agent workload is the expensive, side-effecting case — the story the pitch needs	`--llm=mock`, deterministic
Compute	**2× DigitalOcean droplets**	"Two machines" has to be literally true	two processes, two ports, one laptop
Telemetry	Tiger Data (optional)	Time-series for the migration charts	`--metrics=pg`

The rule that makes this safe: every sponsor sits behind an interface with two implementations, chosen by flag: `mem ⇔ pg`, `fs ⇔ spaces`, `mock ⇔ gemini`. One broken dependency degrades the demo; it cannot kill it.

Prize surface — one build, several categories

SPONSOR / TRACK	HOW THE PROJECT QUALIFIES	PRIZE
YugabyteDB — "Outstanding AI Agent Innovation"	Migrating a *running* AI agent between machines, with strongly consistent fencing so a killed node neither duplicates spend nor loses the run. Their problem, our system.	`₹50,000 + gadgets`
DigitalOcean (MLH)	The two workers are real droplets; chunks live in Spaces	`Robot dog kit`
Gemini API (MLH)	Agent workload's step = one model call, recorded in the op log	`Kreo keyboard`
Tiger Data (MLH)	Migration telemetry (pause, bytes, dedupe) as a time series	`Stream Deck`
Vultr (MLH)	Spare node / cold standby	`Arduino kit`
IOMETE, MongoDB Atlas, Solana, n8n, ElevenLabs, Mastra	**Skip.** No honest fit; forcing one costs hours and convinces no one	–

For the sponsor slide: four logos maximum — YugabyteDB, DigitalOcean, Gemini, Tiger Data. A wall of eleven sponsor logos reads as shopping-list hacking and dilutes the one track we can actually win.

The live demo, second by second

Five minutes. Screen, clicks and narration are all scripted, because the difference between a good demo and a bad one is rehearsal, not code.

TIME	ON SCREEN	SAY
0:00	Dashboard. Two node cards. An agent job running, `seq 47/60`, epoch 0, ops climbing	"This is an AI agent mid-run. Every step is a model call that has already been paid for."
0:30	Point at the five numbers: epoch, seq, ops, bytes, pause	"Five numbers we will come back to: who owns it, how far it got, how much work is durable, how much state moved, and how long the move took."
1:00	Press **Migrate** → **B**. Phase label changes: quiescing → fencing → restoring	"Freeze at the step boundary, ship only the chunks that changed, then move ownership."
1:40	Owner flips to **B**, epoch 0 → 1, seq continues from where it stopped	"Pause: 214 milliseconds. Same job, same sequence number, different machine."
2:20	Migrations table: chunks sent vs deduped	"The second checkpoint shipped one chunk out of seven — the delta is real, and it is measured."
2:50	Kill the source node (or `docker stop` / terminate the droplet)	"Source is gone. Nothing double-executed, because the fenced-out node's writes no longer match its epoch."
3:30	Show a **failed** migration row (`failed-failover-back`)	"This is the failure we rehearsed: after the fence, before restore. Authority went back to the source. Exactly one owner at every instant."
4:10	Job completes; op count equals step count; control run checksum matches	"Eighty steps, eighty committed effects, zero duplicates, and the checksum is identical to a run that never migrated."
4:40	Limitations slide, then sponsor slide	"We move application state, not kernel memory — that is CRIU's problem, and it is a different trade-off. Built on YugabyteDB, DigitalOcean, Gemini."

Demo pre-flight (90 seconds, before you walk on stage)

- Both droplets report in `/api/state`; a long agent job is already running.
- The failure row from the rehearsal is still in the migrations table.
- Dashboard open full-screen on the projector, browser zoom 110%, no other tabs.
- Hotspot connected; **not** campus Wi-Fi. The pre-recorded fallback video is on the desktop.
- Terminal closed. Nothing that can scroll away or show a stack trace.

> IF THE DEMO DIES
>
> Switch to the recording without apology: "Network at the venue is dropping; here is the same run from an hour ago." Then keep narrating over the video. Judges forgive infrastructure; they do not forgive improvisation.

What is built, and what has been measured

This table is the boundary of what may be claimed on a slide. Anything not in the left two columns is "planned" and must be labelled as such.

CAPABILITY	STATUS	EVIDENCE
Step engine, checkpoint, restore, resume	**Built · proven**	`e2e test`
Mid-flight migration across two HTTP workers	**Built · proven**	`epoch 0→1, owner A→B`
Exactly-once effects after migration	**Built · proven**	`ops == steps (800/800)`
Checksum equality with a control run	**Built · proven**	`test assertion`
Fence contention (two callers, one winner)	**Built · proven**	`store test`
Delta transfer (chunks skipped)	**Built · proven**	`1 of 7 chunks`
Dashboard, SSE, one-click migrate	**Built · proven**	`live screenshot`
YugabyteDB / Postgres store	**Written · not executed**	`no DB on dev box`
Spaces blob backend	**Written · not executed**	`credentials pending`
Gemini agent workload	**Written · mock proven**	`API key pending`
Two-droplet deployment	**Planned**	`runbook written`
Automatic recovery of a dead owner	**Gap**	`~2 h of work`

Code facts for the "this is real" slide

GO

3,254

lines across 19 files

PACKAGES

6

workload · node · coordinator · store · chunk · blob

TEST SUITES

3

chunk · store · e2e (4 cases)

Measured runs (verbatim from the logs)

```
# end-to-end test: migrate mid-flight, compare with a control run
migrated job-...  A -> B  epoch 0->1  pause 19 ms  checksum 533443388bb07908
PASS  TestMigrateMidFlightKeepsExactlyOnceSemantics
      op log has 80 rows for 80 steps

# live binary, two workers, real HTTP, 800-step job
-> job job-187ccc596b52 started
-> migrating job-187ccc596b52 from A to B
   outcome=ok epoch=0->1 pause=3ms bytes=13645 chunks_sent=8 deduped=6
-> done seq=800/800 epoch=1 ops=800 checksum=e2a580b9471db5b1
   exactly-once: one committed op per step

# chunking: consecutive checkpoints of the same job
state size=12377  chunks=7  uploaded=7
state size=12377  chunks=7  uploaded=1  skipped=6  bytes=1268
```

Reproduce it in sixty seconds

```
# 1. the automated proof: creates a job, migrates it mid-flight, asserts exactly-once
bash scripts/demo.sh

# 2. the same thing by hand, against the dashboard
go run ./cmd/stateshift --role=all --db=mem --blob=fs --llm=mock
#   -> open http://127.0.0.1:9100, press "Start job", then "Migrate"
```

Both run with zero external services: in-memory control plane, filesystem chunk store, mock model. Deliberate — the numbers have to be reproducible on a laptop with no network.

> **Expected on real infrastructure:** loopback pause is 3–7 ms. Two droplets with a 12 KiB state should land in the **150–600 ms** range, dominated by the checkpoint upload. Say "sub-second", never "three milliseconds", when talking about the deployed demo.

Fourteen slides: copy, data, visual, narration

Five minutes ≈ 20 seconds per slide. Do not exceed three bullets per slide; if a slide needs four, it is two slides. Every number below comes from section 11.

SLIDE 01 · TITLE

CONTENT

StateShift — move a running workload between machines.
Team name + three names. One line under the title: "No restart. No repeated work."

VISUAL

Black type on off-white, one red word in the title. Nothing else. No hero illustration.

SAY "We'll show you a job moving from one machine to another while it's running — and prove nothing was lost or repeated."

SLIDE 02 · THE FAILURE

CONTENT

Three bullets, each a real loss:

- An agent run: 40 tool calls, $6 of tokens, box dies at call 31
- A simulation: six hours of in-memory progress
- A session: state nobody persisted

VISUAL

One horizontal timeline per bullet with a red "X" where it dies and grey "lost work" after it.

SAY "Restarting isn't just slow. It pays twice — for tokens, for compute, for the same work."

SLIDE 03 · WHY THE OBVIOUS ANSWERS FAIL

CONTENT

Redeploy	loses memory, duplicates effects
Save after every step	pins the job to one machine
CRIU	root + matching kernels; all-or-nothing in 24 h
VM live migration	you'd have to own the hypervisor

VISUAL

Four-row table, red tick marks in the "misses" column. Keep it boring and factual.

SAY "Each of these solves part of it. None of them moves a live job between two ordinary machines in a way an application team can use."

SLIDE 04 · THE IDEA

CONTENT

One sentence: **the workload is a state machine; we move its state at a step boundary and hand ownership with a database fence.**

- Checkpoint the state you own, not the kernel state you don't
- Transfer only the bytes that changed
- Exactly one node may hold the epoch

VISUAL

Three boxes left→right: freeze → transfer delta → resume, with a lock icon on the middle one.

SAY "The trick isn't moving memory. It's deciding, atomically, who is allowed to write."

SLIDE 05 · ARCHITECTURE MUST APPEAR

CONTENT

The diagram from section 4, cleaned up: coordinator, two workers, control DB, blob store. Label the arrow directions (register/start, quiesce, restore, chunk PUT/GET).

VISUAL

Redraw in Figma/Keynote — do not paste ASCII. Thin 1px lines, square boxes, one red arrow for the migration path, grey for the data plane. Footnote: control plane and DB never live on a worker.

SAY "Three roles. The coordinator drives; the database decides; the workers execute."

SLIDE 06 · THE FENCE MUST APPEAR

CONTENT

One SQL statement, large, monospaced, on the slide:

```
UPDATE jobs SET epoch = epoch + 1, owner = $target
 WHERE id = $job AND epoch = $current;
```

Caption: "Rows affected must be exactly 1. There is no second winner."

VISUAL

Code, big. Do not add decoration. Optionally two small node icons with one crossed out.

SAY "A heartbeat is a guess that arrives late. This is a predicate evaluated at commit time, on the only copy of the truth."

SLIDE 07 · EXACTLY-ONCE EFFECTS

CONTENT

```
INSERT INTO oplog (job_id, seq, op_id, result)
     VALUES (...) ON CONFLICT DO NOTHING;
```

- Effect log and state advance in one transaction
- Replaying a step is a no-op by schema, not by policy

VISUAL

Two arrows converging into "one transaction". Keep the SQL on screen; it is the argument.

SAY "If a step ever runs twice, the second insert does nothing. Duplication becomes impossible instead of unlikely."

SLIDE 08 · DELTA TRANSFER MUST APPEAR

CONTENT

Big number: **1 of 7 chunks**. Sub-line: 12.4 KiB state, second checkpoint uploads 1,268 B — about 10% of the bytes.

- Content-defined chunking, ~1 KiB average, sha256 keys
- Chunks the store already has are skipped

VISUAL

Seven small squares; six grey (skipped), one red (uploaded). Instantly legible.

SAY "The second snapshot of the same job ships one chunk out of seven. That ratio is the whole transfer-efficiency requirement."

SLIDE 09 · LIVE DEMO MUST APPEAR

CONTENT

Not a slide — a live switch to the dashboard. Have this slide behind you while it runs: five numbers labelled (epoch, seq, ops, bytes, pause).

VISUAL

Five-column stat strip, identical to the dashboard's, so the projection and the product agree.

SAY (during the run) "Quiescing… fencing… restoring. Owner changed, epoch changed, the job did not restart."

SLIDE 10 · FAILURE HANDLING MUST APPEAR

CONTENT

Three rows only:

Dies before the fence	source keeps running
Dies after fence, before restore	authority returns, new epoch
Both alive, partitioned	only the epoch holder commits

VISUAL

Three-state diagram: draining → restoring → running, with the abort arrows drawn in red and labelled with the outcome strings from the migrations table.

SAY "Every failure path ends with exactly one owner. And we log the failures we rehearsed — you'll see one in the table."

SLIDE 11 · EVIDENCE

CONTENT

The two verification lines, monospaced, verbatim:

```
ops == steps            800 / 800
checksum == control     e2a580b9471db5b1
pause                   3 ms  (loopback)
```

Caption: "A migrated run and a run that never migrated end at the same checksum. That's the correctness proof, not a screenshot."

VISUAL

Terminal aesthetic, white on black or black on off-white, no syntax colouring.

SAY "This is the line that matters: the op log has one row per step, and the final checksum is identical to a run that never moved."

SLIDE 12 · LIMITATIONS MUST APPEAR

CONTENT

- We move **application state**, not sockets, file descriptors or kernel memory
- Workloads must express progress as a step machine
- A node dying mid-run without a migration request is not yet auto-recovered (~2 h of work)

VISUAL

Plain text, no icons, generous margins. Deliberately unglamorous — that's the point.

SAY "Transparent process migration is a different problem with different constraints. We chose the version that can be correct in a day, and we'd rather be exact about the boundary than vague about it."

SLIDE 13 · WHAT'S NEXT

CONTENT

- Watchdog: recover a dead owner automatically (designed, not built)
- Streaming transfer instead of upload-then-download
- Third-party workloads via an out-of-process SDK

VISUAL

Three numbered items, one line each. No roadmap Gantt chart.

SAY "The next hour of work is the watchdog. The next week is making it a platform."

SLIDE 14 · CLOSE + SPONSORS MUST APPEAR

CONTENT

One sentence: "StateShift moves a running workload between machines and proves, with a fence and an op log, that nothing was repeated." Plus the four sponsor logos: YugabyteDB · DigitalOcean · Gemini · Tiger Data. Repo URL bottom corner.

VISUAL

Statement slide. Type only, then a thin logo row at the bottom, 22 mm tall, greyscale.

SAY "One machine to another, mid-run, without paying twice. Built on YugabyteDB, DigitalOcean and Gemini. The repo is public — go read the fence."

Numbers you must be able to say without looking

NUMBER	WHAT IT MEANS	HOW TO SAY IT
3–7 ms	Measured pause, loopback, 12 KiB state	"Single-digit milliseconds on one machine"
150–600 ms	Expected pause on two droplets (not yet measured)	"Sub-second across machines" — label it as expected, not measured
1 / 7	Chunks re-uploaded on the second checkpoint	"One chunk out of seven — about ten percent of the bytes"
12.4 KiB	State size in the benchmark	"Twelve kilobytes of state per checkpoint"
800 / 800	Ops committed vs steps, after a mid-flight migration	"Eight hundred steps, eight hundred effects, zero duplicates"
0	Repeated committed work	"We assert it in a test, not hope for it"
2	Nodes needed for the whole demo	"Two cheap droplets"
4	Tables in the control plane (jobs, oplog, checkpoints, migrations)	"Four tables, one of which is the whole consistency story"
3,254	Lines of Go in the core	"A bit over three thousand lines, six packages"
~1 KiB	Average chunk size	"Around a kilobyte per chunk, content-defined"

Numbers to avoid

- **"Zero downtime"** — false; we have a measurable pause. Say "bounded pause".
- **"Scales to N nodes"** — untested. Say "two workers today, the fence generalises".
- **"Transparent to the application"** — the workload implements a step interface. Say "transparent to clients, explicit for the workload".
- **Any percentage of "cost saved"** — we have no billing data. Use the mechanism, not a fabricated saving.

Twelve questions you will get, and the answers

1 · "Is this just Kubernetes?" REHEARSE

No. Kubernetes reschedules a *container*; the process starts again from zero. We keep the workload's progress. A Kubernetes pod that is rescheduled is exactly the failure case we are designed to survive.

2 · "Why not CRIU / real process migration?" REHEARSE

Because it is all-or-nothing and environment-bound: root, kernel support, matching images, and checkpointing sockets and FDs. In a day it either works completely or you demo nothing, and it still cannot migrate arbitrarily. We chose the version that can be correct and measured. It is a deliberate scope boundary, written on the limitations slide.

3 · "What happens if the source never dies — just partitions?" REHEARSE

It becomes a fenced-out writer. Its next commit has `WHERE epoch = 0`, the row now says `epoch = 1`, zero rows are affected, and it stops with no side effects. Correctness does not depend on it noticing anything.

4 · "How is this exactly-once and not at-least-once?" REHEARSE

Because effects carry stable IDs and are inserted with `ON CONFLICT DO NOTHING` in the same transaction that advances the sequence number. A replay cannot re-apply an effect, and the state cannot advance without the effects. It's structural, not a policy.

5 · "Isn't this Temporal / durable execution?"

It is the same family, and we borrow the idea: persist progress at step boundaries. Temporal keeps the job pinned to a worker and replays history to recover. We physically relocate the running job to another machine and measure the move — that's the part they don't ship.

6 · "How big a state can you move?"

The demo moves ~12 KiB. The mechanism is size-agnostic — chunks are streamed through the object store — but we have not benchmarked large states, and pause grows with the delta size. Saying "we haven't measured beyond kilobytes" is the honest answer.

7 · "Is the coordinator a single point of failure?"

Yes, and it is the honest design boundary: the coordinator is stateless apart from its fleet registry, and **the database holds all authority**. A restarted coordinator reads the epoch and continues. Replicating the coordinator was deliberately out of scope for 24 hours.

8 · "What about non-idempotent side effects — an email, a payment?"

Either the effect is recorded through the op log (so it is applied once by construction), or the workload must be pure between checkpoints. We don't claim to make arbitrary side effects safe; we give you the log that makes them safe if you route them through it.

9 · "Why YugabyteDB and not Postgres?"

The fence is a distributed-systems primitive — ownership across machines — so putting it in a distributed SQL database is the coherent choice, and it survives the loss of a database node. The SQL is plain Postgres, which is why the same code runs on Postgres as an offline fallback.

10 · "What did the pause consist of?"

Checkpoint upload, the fence statement, the download and the hash verification. It excludes the current step, which is allowed to finish — that is why a 12 KiB state gives single-digit milliseconds on one machine.

11 · "What if the target rejects the restore?"

Authority returns to the source with another epoch bump, the source restarts from the last committed checkpoint, and the attempt is recorded as `failed-failover-back`. There is always exactly one owner, including during the failure.

12 · "What's the biggest thing missing?"

A watchdog. Today, failures during a migration are adjudicated correctly; a node that dies while owning a job with no migration in flight is not yet auto-recovered. It is a reaper loop, roughly two hours, and we would rather name it than hide it.

How the deck should look

The product is a systems tool; the deck should read like engineering documentation, not a startup pitch. Dense, precise, unglamorous. That aesthetic also happens to be the team's house style — and it photographs well on a projector.

TOKEN	VALUE	USE
Background	`#FAFAF9`	Every slide. Never pure white, never dark.
Ink	`#111111`	All body text and rules.
Muted	`#5E5E59`	Captions, labels, secondary numbers.
Accent	`#E53E3E`	One word per slide, the red arrow, the "fails" mark. Nothing else.
Rule / border	`#DCDCD6 at 1 px`	Table rules, diagram connectors.
Display / body	`Inter`	Everything human-readable. Weights 400 / 600 / 800.
Numbers / code	`JetBrains Mono`	Every number, every SQL snippet, every label.
Grid	`12 columns, 24 px gutter`	Content flush left, never centred.
Corners	`0 px radius`	Sharp. No pills, no cards with shadows.

Layout rules

- **Left-aligned everything.** No centred hero layouts, no centred titles.
- **Maximum three bullets**, maximum two lines each. If it doesn't fit, the slide is wrong.
- **Numbers in mono, at least 40 pt** when they are the argument (1/7, 800/800, 3 ms).
- **No stock photography, no icons except state markers** (a red × and a green square are enough).
- **No animation** beyond a single fade. The demo is the motion.
- **One red element per slide.** It is how the eye finds the claim.

What kills a hackathon deck

ANTI-PATTERN	WHY IT COSTS YOU
A screenshot of 200 lines of code	Unreadable at distance; signals "we could not summarise our own idea"
Eleven sponsor logos	Reads as shopping-list hacking; dilutes the one track you can win
"Revolutionary / seamless / world's first"	Unfalsifiable language invites the question you can't answer
An architecture diagram with 20 boxes	Judges stop reading at box six. Three roles is the right count
No numbers	The competing team with a measured pause time wins on the one axis you left empty
Reading the slides aloud	Wastes 40% of the time you need for the demo

Who does what, and in which order

ROLE	OWNS	DONE BY
A	Go core: job loop, checkpoint/restore, chunker behaviour, the mid-flight path	built
B	Control plane: epoch fence, op log, recovery paths, `/api/migrate`	built
C	Agent workload, Gemini wiring, droplets, Spaces, deployment runbook	scripted, needs credentials
D	Deck, dashboard copy, demo video, pitch delivery	in progress

Order of work on the day (24 hours from 12:00)

CLOCK	GOAL	GATE
12:00–13:00	Accounts, keys, droplets, DB, bucket; deploy the existing binary	both workers register
13:00–15:00	First real migration on real infrastructure; fix DSN/TLS/SDK issues	epoch 0→1 on two droplets
15:00–16:00	Gemini agent workload live; pacing so a job lasts minutes	agent job completes
16:00–18:00	Failure rehearsal: kill the source mid-run, kill during transfer, record a failed row	one `failed-failover-back` visible
18:00–20:00	Watchdog (dead-owner reaper) if time allows	job resumes after a kill
20:00–22:00	Deck assembly + dashboard copy pass	all 14 slides filled
22:00	Mentor session: pitch the working system, collect objections	notes taken
22:00–01:00	Record the demo video while everything is known-good	mp4 exists
01:00–03:00	Sleep rotation: two sleep, two on watch. Non-negotiable	—
03:00–08:00	Polish numbers, tighten deck, second rehearsal	pitch under 5:00
08:00–11:00	Submission text, repo tidy, README numbers refreshed	submitted
11:00–12:00	Buffer. Do not touch code	—

Risk table

RISK	LIKELIHOOD	MITIGATION
Campus Wi-Fi isolates the two laptops	high	Hotspot; demo on droplets
Yugabyte YSQL edge case in the schema	medium	`--db=pg` fallback; worst case one SQL statement rewritten
Spaces SDK / bucket config	medium	`--blob=fs`, and say so
Gemini rate limit mid-demo	medium	`--llm=mock`, same op-log story
Someone chases CRIU at 3 AM	high	Written rule: never on the demo path
Demo machine sleeps / locks	medium	Power settings off; video fallback

Reference material

API surface

METHOD	PATH	PURPOSE
GET	/	embedded dashboard
GET	/api/health	liveness
GET	/api/state	snapshot: nodes, jobs, migrations, totals
GET	/api/events	SSE stream feeding the dashboard
POST	/api/jobs	create + start a job
POST	/api/migrate	run a handover
POST	/api/nodes/register	worker heartbeat
POST	/internal/start	*worker*: begin or resume a job
POST	/internal/quiesce	*worker*: freeze at a step boundary + checkpoint
POST	/internal/restore	*worker*: pull, verify, resume

Schema (four tables)

TABLE	HOLDS	KEY INVARIANT
jobs	epoch, seq, status, owner, state, state_hash, pause, bytes, checksum	one row per job; the epoch lives here
oplog	(job_id, seq, op_id) → result	primary key makes a replay a no-op
checkpoints	(job_id, seq) → manifest key, state hash, chunk stats	a checkpoint exists only if its manifest exists
migrations	from, to, epochs, timestamps, pause, bytes, chunks, outcome	every attempt recorded, including failures

Commands that produce the demo

```
# single machine, two workers, zero dependencies
go run ./cmd/stateshift --role=all --db=mem --blob=fs --blob-dir=.run/blob --llm=mock

# the automated proof (creates, migrates, asserts exactly-once)
bash scripts/demo.sh

# deploy: build on the laptop, run on Linux
make linux

# two droplets
./stateshift --role=api  --listen :9100 --db=yugabyte --blob=spaces
./stateshift --role=node --listen :9101 --node-id droplet-1 --coordinator http://COORD:9100
./stateshift --role=node --listen :9102 --node-id droplet-2 --coordinator http://COORD:9100
```

Glossary — if a judge uses one of these words

Epoch	A monotonic ownership counter. Whoever holds the current epoch may write; everyone else is stale.
Fence	The single guarded UPDATE that increments the epoch and moves ownership. Exactly one caller can win it.
Quiesce	Stop starting new work, finish the step in flight, checkpoint, then stop executing.
Op log	Append-only record of committed effects, keyed by (job, seq, op_id) so replays are no-ops.
CDC	Content-defined chunking: chunk boundaries follow the data, so edits shift only nearby chunks.
Manifest	The checkpoint envelope: chunk list plus the hash the reassembled state must match.
Delta	The bytes actually moved, which is the chunks the store does not already have.
Pause	Quiesced → resumed interval. The headline number.

`StateShift · internal brief · generated from the working repository (commit 6a16485) · every metric in this document was produced by an executed test or a live run, except where explicitly labelled expected or planned`